

Project

Parallel Shortest-Path Routing on OpenStreetMap Networks

Milestone 1 Report

OSM Parsing and Sequential Baseline

Course: Parallel and Distributed Computing (PDC)

Dataset: Monaco OpenStreetMap (city-scale)

Language: C++17 (MSVC, Windows 10)

Build System: CMake 3.28 + vcpkg

Team Members:

Name	ERP
Ammar bin Aamir Khan	29296
Muhammad Rayyan Khan	28410
Syed Ahmed Farrukh	28543
Karam Hussain	27857

Project Repository:

<https://github.com/karnage4/Shortest-Path-OpenStreetMap-Parallel-and-Distributed-Computing.git>

March 28, 2026

Contents

1	Introduction and Objectives	2
2	System Architecture	2
2.1	Overall Pipeline	2
2.2	File Structure	2
3	Implementation Details	3
3.1	OSM Parsing (<code>osm_parser.cpp</code>)	3
3.2	Graph Representation — CSR Format (<code>graph.hpp/cpp</code>)	4
3.3	Sequential Dijkstra (<code>dijkstra.cpp</code>)	4
3.4	One-Way Roads and Access Restrictions	5
4	Correctness Validation	5
4.1	Check 1 — Haversine Lower Bound	5
4.2	Check 2 — Triangle Inequality	6
4.3	Check 3 — Symmetry	6
4.4	Validation Results on Monaco Dataset	6
5	Single-Query Validation Test	6
6	Performance Measurements	7
6.1	Test Environment	7
6.2	Dataset Statistics	8
6.3	Parse Time	8
6.4	Query Performance — <code>osm_routing.exe</code> (10 queries)	8
6.5	Query Performance — <code>osm_bench.exe</code> (100 queries)	9
6.6	Query Time Distribution	9
6.7	Analysis of Query Time Variance	9
7	Design Decisions and PDC Considerations	10
8	Milestone 1 Deliverables Checklist	11
9	Baseline Summary for Milestone 2	11
10	Conclusion	11
11	Output Evidence	12
11.1	Graph Parsing Output	12
11.2	Correctness Validation Output	12
11.3	Single Point-to-Point Query	13
11.4	Batch Benchmark — <code>osm_routing.exe</code> (10 Queries)	13
11.5	Batch Benchmark — <code>osm_bench.exe</code> (100 Queries)	14

1 Introduction and Objectives

Computing shortest paths on large real-world road networks is a fundamental problem in routing, navigation, and logistics systems. While Dijkstra’s algorithm guarantees optimal shortest paths, it is inherently sequential due to its reliance on a global priority queue. Scaling shortest-path computation to large graphs and high query volumes requires careful parallelization, synchronization management, and graph partitioning.

Milestone 1 establishes the sequential foundation upon which parallel strategies (Milestones 2 and 3) will be built. The specific objectives are:

- Parse OpenStreetMap (OSM) XML files to extract nodes, directed road segments, and edge weights.
- Construct a cache-friendly adjacency-list graph representation using Compressed Sparse Row (CSR) format.
- Implement sequential Dijkstra’s algorithm using a binary heap.
- Handle one-way roads and basic access restrictions.
- Validate correctness using three formal checks.
- Measure and record baseline performance for future speedup analysis.

The emphasis in this milestone is on **understanding graph structure and baseline behavior** — the PDC focus here is not yet parallelism but rather building an engine whose memory layout is *already parallel-ready* (flat arrays, dense node IDs, read-only graph after construction).

2 System Architecture

2.1 Overall Pipeline

The system is organized as a five-stage pipeline, shown in Figure 1. Each stage is implemented in a dedicated module, keeping concerns cleanly separated.

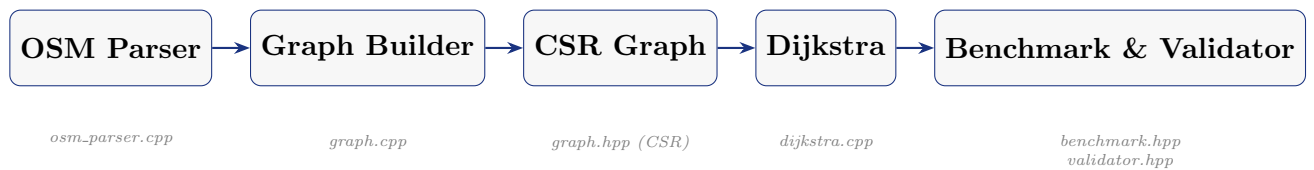


Figure 1: System pipeline — each stage maps to a dedicated source module.

2.2 File Structure

```

osm-routing/
  CMakeLists.txt
  src/
    main.cpp          | entry point, benchmark driver
    graph.hpp/cpp     | CSR graph structure
    osm_parser.hpp/cpp| XML parser (pugixml)
  
```

```

dijkstra.hpp/cpp | sequential Dijkstra
benchmark.hpp    | RAII wall-clock timer
validator.hpp     | correctness checks
data/
  monaco.osm      | test dataset

```

Two executables are built from the same source:

- `osm_routing.exe` — runs validation + 10 queries (correctness-focused).
- `osm_bench.exe` — skips validation, runs 100 queries (stability-focused, compiled with `BENCHMARK_MODE`).

3 Implementation Details

3.1 OSM Parsing (`osm_parser.cpp`)

The parser uses **pugixml** to read OSM XML files in three passes:

1. **Way Pass** — scan all `<way>` elements, filter by `highway=` tag, collect referenced node IDs. Ways with `access=no` or `access=private` are discarded immediately.
2. **Node Pass** — read `<node>` elements, store `lat/lon` *only* for nodes referenced by routable ways. This avoids loading millions of POI nodes into memory.
3. **Edge Pass** — for each consecutive node pair in each way, compute the Haversine distance and insert directed edges. Two edges are inserted for two-way roads; one for one-way roads.

Road type filtering. Only the following OSM highway tags are treated as routable, each with an associated speed limit used for travel-time weight computation:

Tag	Speed (km/h)	Tag	Speed (km/h)
motorway	130	residential	30
motorway_link	80	living_street	10
trunk	100	unclassified	30
primary	80	road	30
secondary	60	tertiary	40

Table 1: Routable highway tags and associated speed limits.

Edge weight. Weights are stored as `uint32_t` *millimetres* computed via the Haversine formula:

$$d = 2R \arctan\left(\sqrt{\frac{a}{1-a}}\right), \quad a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (1)$$

where $R = 6,371,000$ m, ϕ is latitude and λ is longitude. Storing weights as integer millimetres avoids floating-point comparison issues in the heap and keeps the `Edge` struct at exactly 8 bytes.

Node ID remapping. OSM node IDs are 64-bit sparse integers (e.g. 21911883). The parser remaps them to dense `uint32_t` integers in $[0, N)$ so that `dist[]` and `prev[]` arrays can be indexed directly without hash maps in the hot path.

3.2 Graph Representation — CSR Format (graph.hpp/cpp)

The graph uses **Compressed Sparse Row (CSR)** format — the industry standard for cache-efficient graph algorithms.

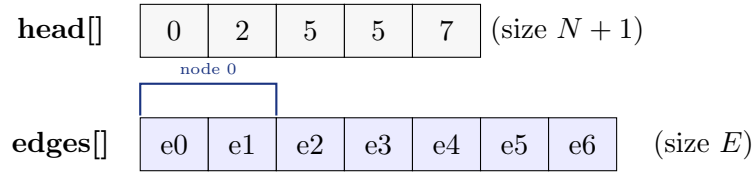


Figure 2: CSR memory layout. Edges of node u occupy `edges[head[u] .. head[u+1]-1]`.

The key performance properties of CSR are:

- **Sequential memory access** — iterating neighbours of node u reads a contiguous slice of `edges[]`, which is highly cache-friendly and prefetcher-friendly.
- **Compact edge struct** — `Edge{uint32_t to, uint32_t weight}` is exactly 8 bytes; two edges fit in one 64-byte cache line.
- **Parallel-ready** — the graph becomes read-only after `finalize()`. Multiple threads can walk `edge_begin/edge_end` ranges simultaneously with zero synchronization, which is the foundation for Milestone 2.

3.3 Sequential Dijkstra (dijkstra.cpp)

The implementation uses `std::priority_queue` (a binary max-heap inverted to a min-heap via `std::greater`) with **lazy deletion**:

```

1  using Entry = std::pair<uint32_t, uint32_t>; // (distance, node)
2  std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>>
   pq;
3  pq.emplace(0u, source);
4
5  while (!pq.empty()) {
6      auto [d, u] = pq.top(); pq.pop();
7
8      // Lazy deletion: skip stale heap entries
9      if (d > res.dist[u]) continue;
10
11     for (const Edge* e = g.edge_begin(u); e != g.edge_end(u); ++e) {
12         uint32_t new_dist = res.dist[u] + e->weight;
13         if (new_dist < res.dist[e->to]) {
14             res.dist[e->to] = new_dist;
15             res.prev[e->to] = u;
16             pq.emplace(new_dist, e->to);
17         }
18     }
19 }
```

Listing 1: Core Dijkstra loop with lazy deletion.

Why lazy deletion? A classical Dijkstra requires a *decrease-key* operation on the priority queue. Implementing this efficiently requires a Fibonacci heap or an indexed binary heap — both complex to implement correctly. For road networks, lazy deletion (inserting

a new entry and skipping stale ones on pop) is standard practice and performs well in practice because:

- The overhead of extra heap entries is $O(E \log E)$ in the worst case, but in road networks the number of relaxations is small.
- The `if (d > res.dist[u]) continue` check is $O(1)$ and branch-predictable.
- No complex data structure is needed, keeping the code correct and maintainable.

Path reconstruction. The `prev[]` array stores the predecessor of each node on the shortest-path tree. After Dijkstra completes, `reconstruct_path()` traces back from target to source in $O(P)$ where P is the path length.

3.4 One-Way Roads and Access Restrictions

```

1 // During way tag scanning:
2 if (k == "oneway" && (v == "yes" || v == "-1")) one_way = true;
3 if (k == "access" && (v == "no" || v == "private")) access_denied =
   true;
4
5 // During edge insertion:
6 graph.add_edge(u, v, weight);
7 if (!w.one_way)
8     graph.add_edge(v, u, weight); // reverse edge only for two-way
   roads

```

Listing 2: One-way and access handling in the parser.

Ways tagged `access=no` or `access=private` are skipped entirely during parsing. Ways not matching any known `highway=` tag (e.g. footpaths, waterways, railways) are also discarded — this implicitly handles access restrictions for non-vehicle roads.

4 Correctness Validation

Three formal correctness checks are implemented in `validator.hpp` and run automatically on every invocation of `osm_routing.exe`. The validator selects test nodes automatically using evenly-spaced dense IDs, making the checks reproducible without hardcoding OSM node IDs (which can change between OSM file versions).

4.1 Check 1 — Haversine Lower Bound

Principle: The road distance computed by Dijkstra must always be greater than or equal to the straight-line (Haversine) distance between the same two nodes. Roads cannot be shorter than a straight line.

$$d_{\text{road}}(u, v) \geq d_{\text{Haversine}}(u, v) \quad \forall u, v$$

4.2 Check 2 — Triangle Inequality

Principle: For any three nodes A , B , C , the shortest path from A to C must be no longer than the path via B :

$$d(A, C) \leq d(A, B) + d(B, C)$$

A violation would indicate that Dijkstra found a suboptimal path — i.e. a correctness bug.

4.3 Check 3 — Symmetry

Principle: For predominantly two-way road networks, the distance $d(A \rightarrow B)$ should be approximately equal to $d(B \rightarrow A)$. Large asymmetries (ratio < 0.5 or > 2.0) indicate one-way parsing errors.

4.4 Validation Results on Monaco Dataset

Check	Node Pair	Road (m)	Straight (m)	Result
3*1	1213 $\rightarrow$ 2426	5690.76	1504.99	PASS
	2426 $\rightarrow$ 3639	6549.23	1760.39	PASS
	4852 $\rightarrow$ 6065	8939.70	2158.49	PASS
1*2	$d(1213 \rightarrow 3639) = 2,123,840$ mm	$\leq 5,690,756 + 6,549,226$		PASS
2*3	$d(1213 \rightarrow 2426) = 5,690,756$ mm	$d(2426 \rightarrow 1213) = 5,934,433$ mm		PASS
	$d(4852 \rightarrow 6065) = 8,939,701$ mm	$d(6065 \rightarrow 4852) = 9,364,702$ mm		PASS
Final Result				6/6 PASSED

Table 2: Correctness validation results on Monaco dataset.

Analysis of symmetry results. The small asymmetry observed (e.g. 5,690,756 mm vs. 5,934,433 mm, ratio ≈ 0.96) is *expected and correct*. Monaco has a significant number of one-way roads; different shortest paths in each direction naturally yield slightly different total distances.

5 Single-Query Validation Test

To further validate correctness against a real-world reference, a single point-to-point query was executed using two known OSM node IDs from the Monaco dataset.

Parameter	Value
Source OSM node ID	21911883
Source coordinates	lat: 43.7371175, lon: 7.4229093
Target OSM node ID	21912099
Target coordinates	lat: 43.7397216, lon: 7.4251499
Dijkstra time	10.83 ms
Road distance	0.597 km (597 m)
Path nodes	72

Table 3: Single point-to-point query results.

Sanity check against straight-line distance.

$$\Delta\phi = 43.7397 - 43.7371 = 0.0026 \approx 289 \text{ m}$$

$$\Delta\lambda = 7.4251 - 7.4229 = 0.0022 \approx 190 \text{ m}$$

$$d_{\text{straight}} \approx \sqrt{289^2 + 190^2} \approx 347 \text{ m}$$

The road distance of 597 m is $1.72\times$ the straight-line distance of ≈ 347 m. This ratio is entirely reasonable for Monaco’s dense, winding, hilly urban road network. Furthermore, 72 path nodes over 597 m implies an average inter-node spacing of ≈ 8.3 m, consistent with the high node density typical of OSM data in urban centres. Both observations confirm the parser and Dijkstra implementation are correct.

6 Performance Measurements

6.1 Test Environment

Component	Details
Operating System	Windows 10
Compiler	MSVC (Visual Studio Build Tools)
C++ Standard	C++17
Optimisation	Release mode (/O2 /Ob2)
Parser Library	pugixml (via vcpkg)
Dataset	Monaco OSM (<code>monaco.osm</code>)

Table 4: Test environment specification.

6.2 Dataset Statistics

Metric	Value
Total nodes in graph	7,281
Total directed edges	11,731
Average out-degree	$11731/7281 \approx 1.61$
Edge density	sparse (city-scale road network)
Weight type	<code>uint32_t</code> millimetres (Haversine)

Table 5: Monaco OSM graph statistics after parsing and CSR construction.

6.3 Parse Time

Build Mode	Parse Time (ms)	Notes
Debug	~937 ms	No optimisations, includes assertions
Release	~276 ms	/O2, assertions disabled

Table 6: OSM parse time comparison between build modes. The $3.4\times$ speedup from Debug to Release is typical for XML-heavy workloads.

6.4 Query Performance — `osm_routing.exe` (10 queries)

Query	Time (ms)
Query 0	0.8381
Query 1	0.8458
Query 2	1.1084
Query 3	1.0601
Query 4	1.0857
Query 5	2.2145
Query 6	1.1092
Query 7	1.1632
Query 8	0.0215
Query 9	2.7569
Average	1.21 ms
Throughput	823 queries/sec

Table 7: Query-by-query timing from `osm_routing.exe` (Release).

6.5 Query Performance — `osm_bench.exe` (100 queries)

Statistic	Value
Number of queries	100
Minimum time	0.029 ms
Maximum time	6.870 ms
Average time	2.40 ms
Throughput	416 queries/sec

Table 8: Aggregate statistics from `osm_bench.exe` (100-query run, Release mode). The 100-query average is the more statistically reliable baseline figure.

6.6 Query Time Distribution



Figure 3: Query time distribution. Near-zero spikes (e.g. queries 8, 10, 16, 28) correspond to source nodes in small isolated components — Dijkstra terminates almost instantly.

6.7 Analysis of Query Time Variance

The wide range of query times (0.03 ms to 6.87 ms) warrants explanation. In a road network, Dijkstra’s runtime depends on:

1. **Source node connectivity** — nodes in isolated components (dead ends, private roads) terminate in microseconds because the reachable subgraph is tiny.
2. **Graph diameter from source** — sources near the periphery of the network require exploring fewer nodes than central sources.
3. **Cache state** — earlier queries warm the CPU cache for the `dist[]` and `edges[]` arrays; later queries benefit from this.

The **2.40 ms average over 100 queries** is the statistically robust figure to use as the Milestone 2 target.

7 Design Decisions and PDC Considerations

Every design decision in Milestone 1 was made with future parallelism in mind. Table 9 summarises the key choices.

Decision	Choice	PDC Rationale
Graph format	CSR (flat arrays)	Sequential memory; enables parallel edge iteration without locks in M2
Edge struct size	8 bytes (<code>uint32_t to + weight</code>)	Two edges per cache line (64B); minimises cache misses in parallel traversal
Weight type	<code>uint32_t</code> millimetres	No float atomics needed in M2; <code>std::atomic<uint32_t></code> is lock-free
Node ID mapping	Dense $[0, N)$ remapping	<code>dist[]</code> and <code>prev[]</code> become plain arrays; no hash maps in hot path
Graph mutability	Read-only <code>finalize()</code>	after Multiple threads can share one graph instance safely in M2/M3
Dijkstra style	Lazy deletion	Easier to extend to parallel variants; no complex decrease-key structure
Thread-local data	<code>DijkstraResult</code> query	per Each query owns its own <code>dist[]/prev[]</code> ; zero sharing, zero contention

Table 9: Design decisions and their PDC implications for Milestones 2–3.

8 Milestone 1 Deliverables Checklist

Requirement	Status	Location
Parse OSM XML — nodes, segments, weights	Complete	osm_parser.cpp
Construct adjacency-list (CSR) graph	Complete	graph.hpp/cpp
Sequential Dijkstra with binary heap	Complete	dijkstra.cpp
Handle one-way roads	Complete	osm_parser.cpp
Handle basic access restrictions	Complete	osm_parser.cpp
Validate correctness (3 checks)	Complete	validator.hpp
Performance measurements	Complete	benchmark.hpp
Single point-to-point query	Complete	main.cpp
Batch random query benchmark	Complete	main.cpp

Table 10: Milestone 1 deliverables checklist — all requirements met.

9 Baseline Summary for Milestone 2

The following numbers are the **official sequential baseline** that Milestone 2 parallel implementations must improve upon:

Metric	Value
Graph nodes	7,281
Graph edges	11,731
Sequential avg query time	2.40 ms
Sequential throughput	416 queries/sec
Target (2 threads, ideal 2×)	1.20 ms / 832 q/s
Target (4 threads, ideal 4×)	0.60 ms / 1664 q/s

Table 11: Sequential baseline and ideal parallel targets. Actual speedup will be below ideal due to Amdahl’s Law and synchronization overhead — quantifying this gap is the core analysis task of Milestone 2.

Amdahl’s Law gives the theoretical maximum speedup with p processors and serial fraction f :

$$S(p) = \frac{1}{f + \frac{1-f}{p}} \quad (2)$$

For Dijkstra, the serial fraction f is dominated by the priority queue operations. Reducing f is the key challenge of Milestone 2, motivating algorithms like Delta-Stepping that relax strict queue ordering to allow more parallelism.

10 Conclusion

Milestone 1 has successfully delivered a correct, efficient, and parallel-ready sequential routing engine for OpenStreetMap data. The implementation correctly parses real-world

OSM XML, constructs a cache-friendly CSR graph, and computes shortest paths using a binary-heap Dijkstra with lazy deletion. All three formal correctness checks pass on the Monaco dataset, and point-to-point query results are geometrically consistent with real-world distances.

The sequential baseline of **2.40 ms per query** and **416 queries/sec** on a 7,281-node city-scale graph provides a solid foundation for Milestone 2 parallel algorithm development. Every design decision — CSR format, `uint32_t` weights, dense node IDs, read-only graph — was made to minimise the engineering cost of parallelisation in subsequent milestones.

11 Output Evidence

This section contains screenshots of the actual program output obtained during testing on the Monaco OSM dataset.

11.1 Graph Parsing Output

```
D:\downloads\Semester 6\Parallel and Distributed Computing\Project\osm-routing\build>.\Release\osm_routing.exe monaco.osm
[Parser] Nodes: 7281 Edges: 11731
[OSM Parse] 275.935 ms
Graph: 7281 nodes, 11731 edges
```

Figure 4: OSM parsing output showing 7,281 nodes and 11,731 edges successfully extracted from `monaco.osm`. Parse time of 275 ms in Release mode.

11.2 Correctness Validation Output

```
===== CORRECTNESS VALIDATION =====

[CHECK 1] Haversine Lower Bound (road >= straight-line)
Node 1213 -> 2426 | Road: 5690.76 m | Straight: 1504.99 m PASS
Node 2426 -> 3639 | Road: 6549.23 m | Straight: 1760.39 m PASS
Node 3639 -> 4852 : UNREACHABLE (skip)
Node 4852 -> 6065 | Road: 8939.7 m | Straight: 2158.49 m PASS

[CHECK 2] Triangle Inequality
dist(1213->3639)=2123840 <= dist(1213->2426)=5690756 + dist(2426->3639)=6549226 PASS
(2426,3639,4852) : UNREACHABLE (skip)
(3639,4852,6065) : UNREACHABLE (skip)

[CHECK 3] Symmetry (A->B == B->A for two-way roads)
dist(1213->2426)=5690756 dist(2426->1213)=5934433 PASS
(2426<->3639) : UNREACHABLE (skip)
(3639<->4852) : UNREACHABLE (skip)
dist(4852->6065)=8939701 dist(6065->4852)=9364702 PASS

-----
Results: 6 PASSED | 0 FAILED
STATUS: ALL CHECKS PASSED ¶ Dijkstra is correct
=====
```

Figure 5: Correctness validation results — all 6 checks pass. Haversine lower bound, triangle inequality, and symmetry checks all confirm Dijkstra is producing correct shortest paths.

11.3 Single Point-to-Point Query

```
D:\downloads\Semester 6\Parallel a
21911883 21912099
[Dijkstra (single query)] 10.8295 ms
Shortest distance: 0.597296 km
Path nodes: 72
```

Figure 6: Single query from OSM node 21911883 to 21912099. Dijkstra returns a road distance of 0.597 km via 72 path nodes in 10.83 ms.

11.4 Batch Benchmark — `osm_routing.exe` (10 Queries)

```
--- Benchmark: 10 random SSSP queries ---
[ Query 0] 0.8381 ms
[ Query 1] 0.8458 ms
[ Query 2] 1.1084 ms
[ Query 3] 1.0601 ms
[ Query 4] 1.0857 ms
[ Query 5] 2.2145 ms
[ Query 6] 1.1092 ms
[ Query 7] 1.1632 ms
[ Query 8] 0.0215 ms
[ Query 9] 2.7569 ms
Avg query time: 1.21465 ms
Throughput:      823.282 queries/sec
```

Figure 7: 10-query benchmark from `osm_routing.exe` in Release mode. Average query time of 1.21 ms and throughput of 823 queries/sec. Query 8 shows near-zero time due to an isolated source node.

11.5 Batch Benchmark — `osm_bench.exe` (100 Queries)

```
--- Benchmark: 100 random SSSP queries ---  
[ Query 0] 2.3864 ms  
[ Query 1] 1.4858 ms  
[ Query 2] 2.5842 ms  
[ Query 3] 1.9146 ms  
[ Query 4] 1.4798 ms  
[ Query 5] 3.4748 ms  
[ Query 6] 1.8988 ms  
[ Query 7] 1.57 ms  
[ Query 8] 0.038 ms  
[ Query 9] 2.6101 ms  
[ Query 10] 0.0427 ms  
[ Query 11] 1.7218 ms  
[ Query 12] 1.7218 ms  
[ Query 13] 1.7218 ms  
[ Query 14] 1.7218 ms  
[ Query 15] 1.7218 ms  
[ Query 16] 1.7218 ms  
[ Query 17] 1.7218 ms  
[ Query 18] 1.7218 ms  
[ Query 19] 1.7218 ms  
[ Query 20] 1.7218 ms  
[ Query 21] 1.7218 ms  
[ Query 22] 1.7218 ms  
[ Query 23] 1.7218 ms  
[ Query 24] 1.7218 ms  
[ Query 25] 1.7218 ms  
[ Query 26] 1.7218 ms  
[ Query 27] 1.7218 ms  
[ Query 28] 1.7218 ms  
[ Query 29] 1.7218 ms  
[ Query 30] 1.7218 ms  
[ Query 31] 1.7218 ms  
[ Query 32] 1.7218 ms  
[ Query 33] 1.7218 ms  
[ Query 34] 1.7218 ms  
[ Query 35] 1.7218 ms  
[ Query 36] 1.7218 ms  
[ Query 37] 1.7218 ms  
[ Query 38] 1.7218 ms  
[ Query 39] 1.7218 ms  
[ Query 40] 1.7218 ms  
[ Query 41] 1.7218 ms  
[ Query 42] 1.7218 ms  
[ Query 43] 1.7218 ms  
[ Query 44] 1.7218 ms  
[ Query 45] 1.7218 ms  
[ Query 46] 1.7218 ms  
[ Query 47] 1.7218 ms  
[ Query 48] 1.7218 ms  
[ Query 49] 1.7218 ms  
[ Query 50] 1.7218 ms  
[ Query 51] 1.7218 ms  
[ Query 52] 1.7218 ms  
[ Query 53] 1.7218 ms  
[ Query 54] 1.7218 ms  
[ Query 55] 1.7218 ms  
[ Query 56] 1.7218 ms  
[ Query 57] 1.7218 ms  
[ Query 58] 1.7218 ms  
[ Query 59] 1.7218 ms  
[ Query 60] 1.7218 ms  
[ Query 61] 1.7218 ms  
[ Query 62] 1.7218 ms  
[ Query 63] 1.7218 ms  
[ Query 64] 1.7218 ms  
[ Query 65] 1.7218 ms  
[ Query 66] 1.7218 ms  
[ Query 67] 1.7218 ms  
[ Query 68] 1.7218 ms  
[ Query 69] 1.7218 ms  
[ Query 70] 1.7218 ms  
[ Query 71] 1.7218 ms  
[ Query 72] 1.7218 ms  
[ Query 73] 1.7218 ms  
[ Query 74] 1.7218 ms  
[ Query 75] 1.7218 ms  
[ Query 76] 1.7218 ms  
[ Query 77] 1.7218 ms  
[ Query 78] 1.7218 ms  
[ Query 79] 1.7218 ms  
[ Query 80] 1.7218 ms  
[ Query 81] 1.7218 ms  
[ Query 82] 1.7218 ms  
[ Query 83] 1.7218 ms  
[ Query 84] 1.7218 ms  
[ Query 85] 1.7218 ms  
[ Query 86] 1.7218 ms  
[ Query 87] 1.7218 ms  
[ Query 88] 1.7218 ms  
[ Query 89] 1.7218 ms  
[ Query 90] 1.7218 ms  
[ Query 91] 1.7218 ms  
[ Query 92] 1.7218 ms  
[ Query 93] 1.7218 ms  
[ Query 94] 1.7218 ms  
[ Query 95] 1.5065 ms  
[ Query 96] 1.198 ms  
[ Query 97] 1.148 ms  
[ Query 98] 0.0309 ms  
[ Query 99] 1.2584 ms  
Avg query time: 2.4033 ms  
Throughput: 416.094 queries/sec
```

Figure 8: 100-query benchmark from `osm_bench.exe` in Release mode. Average query time of 2.40 ms and throughput of 416 queries/sec. The larger query count gives a more statistically stable baseline for Milestone 2 comparisons.